# Final Report

*Word count: ~1,230 (Parts A and B combined, excluding this line)*

## Part A — Critical Evaluation of the Process

This coursework built a full pipeline from raw r/srilanka posts to policy-relevant insight: collection (50,014 posts) → cleaning and tokenization (47,229 posts, 3 comparable schemes) → silver-standard categorization (10 categories via clustering) → classification across three genuinely different paradigms (fine-tuned encoders, NLI zero-shot, decoder-LLM zero/few-shot prompting) → schema-driven information extraction → LLM-generated multi-stakeholder summarization with an explicit bias assessment. Evaluated honestly, the process was useful in ways that go beyond the specific numbers produced, but it also surfaced concrete, evidenced limitations that matter for anyone considering this kind of pipeline for a real civic application.

**Usefulness.** The clearest value was methodological rather than any single result. Comparing eight classification approaches under an identical test protocol produced a real, transferable finding: fine-tuned BERT/RoBERTa (0.87–0.88 accuracy) beat every zero/few-shot approach by roughly 2.4×, and — less obviously — *model scale mattered more than prompting strategy*: Phi-3-mini zero-shot outperformed the smaller Qwen2.5-1.5B's few-shot results. That is a genuinely useful, generalizable lesson about when fine-tuning is worth its cost, not just a Sri Lanka-specific artifact. Similarly, Task 5's grounding methodology — every summarization claim checked against statistics computed independently of the LLM, plus a second, different model used purely as judge — is a reusable pattern for making any generative pipeline auditable, and proved its worth directly (below).

**Limitations.** Several are structural, not incidental. First, because manual annotation was out of scope, every downstream metric measures *agreement with a fallible label source*, not ground truth: Task 3's classification "accuracy" is agreement with unsupervised clustering labels, and Task 4's fine-tuned NER model (0.38 micro-F1) was trained on labels derived from the LLM's own extractions, which were themselves only 87.5% grounded. The NER result should be read as "how well BERT learned to imitate an imperfect teacher," not independent accuracy. Second, compute constraints forced real subsampling (2,000 of 9,446 test posts for decoder-LLM classification; 500 of 2,583 posts for extraction and summarization) — defensible trade-offs, but they narrow how far any conclusion legitimately extends. Third, and most concretely: the Task 5 summary itself contained a pure fabrication ("custom jewelry," present in zero of 500 source posts, traced to the reduce stage of the summarization pipeline) and a single real post — one person's account of job-seeking as a trans woman — that lost its frequency signal as it propagated through map-reduce and appeared in the final report as an apparent community-wide theme. Neither was caught by the automated numeric fact-checker (100% pass) or by an independent LLM judge (4/5 on "absence of hallucination"). Only manual reading against source data found either issue. That is the single most important empirical result of the whole project.

**LLM potential for public-comment-to-government recommendations.** This directly informs the coursework's own framing: could this kind of pipeline generate recommendations to government from public comments, as a participatory-democracy tool for Sri Lanka? The upside is real: LLMs make it computationally cheap to synthesize sentiment at a scale no human team could read manually, potentially widening whose voices are heard beyond those with the time or access for formal consultation. But this project's own evidence argues against deploying it *directly* today. A policy brief containing an invented business trend is not a hypothetical risk — this project produced one. A citizen's personal, potentially sensitive disclosure being repackaged as "market intelligence" for policymakers, without their knowledge, is not a hypothetical harm — this project did that too, from a real post. And even a perfectly faithful summary of r/srilanka would only represent one constituency: the extracted top skills (python, english, software engineering) and locations (Colombo-heavy) show a subreddit skewing younger, urban, and English-literate. Feeding that to policymakers as if it were general public opinion risks *manufacturing* false consensus, not capturing real consensus — a genuine threat to, not enhancement of, participatory legitimacy. The technology's real near-term role is as a triage and synthesis aid *within* a framework of mandatory grounding verification (as built here), explicit representativeness disclaimers, sensitivity screening before vulnerable disclosures can influence output, and a human reviewer before anything reaches a decision-maker — not as an unsupervised pipeline from comments to policy text.

## Part B — Reflective Report on the Learning Process

*The factual content below reflects what was actually built and how the collaboration worked; the more personal reflections — what genuinely confused me, what clicked, how it felt to debug at 2am — are mine to add and should be read as a draft for me to personalize before submission.*

The five tasks built on each other in a way that mirrored how I'd imagine a real applied NLP project unfolding, not a sequence of isolated exercises. Early tasks (collection, EDA, cleaning) taught foundational discipline in handling messy, real user-generated text at scale — the kind of judgment calls (where to threshold post length, how to justify it with percentiles rather than a round number) that don't come up in clean textbook datasets. Later tasks demanded synthesizing separate pieces of coursework knowledge into single, practical decisions: choosing WordPiece over three close alternatives required actually interpreting perplexity and compression trade-offs together, not just computing them. Task 3 taught me, empirically rather than abstractly, why fine-tuning still matters even in an era of capable zero-shot LLMs. Task 5 taught me the most uncomfortable and valuable lesson: that an LLM output can look completely clean by every automated measure I built — and still be wrong in a way only careful human reading catches.

Generative AI (Claude Code) was used throughout as an active collaborator, not a one-shot code generator, and being honest about that role is presumably the point of this section. It wrote and iteratively debugged every pipeline script, but consequential decisions — which category to focus Tasks 4–5 on, whether to attempt the optional NER fine-tuning comparison, how large a sample was defensible given Colab's GPU limits — were put to me explicitly rather than assumed silently. It also caught concrete problems I likely would have spent much longer finding myself: a training configuration that silently produced an 11GB+ checkpoint dump instead of a

normal one, stale file paths left over after a folder rename that would have broken two scripts on a fresh clone, and a `scikit-learn` edge case that crashed a report script when predictions fell outside the expected label set. More importantly, it built independent verification *into* the process by default — unit testing parsing and scoring logic against synthetic data before ever spending GPU time on Colab, and, in Task 5, deliberately designing an evaluation that was skeptical of its own output (a second, different model as judge; every numeric claim traced back to source statistics).

That last point is the main thing I'm taking away about working with generative AI on technical work: it is a powerful collaborator but not a self-certifying one. The Task 5 hallucinations passing every automated check is direct proof that AI-assisted output — whether it's code or generated analysis — needs explicit, independent verification designed into the workflow, not assumed because the process looks rigorous. Where I'd add my own honest reflection: because so much implementation and debugging happened collaboratively, my own effort this term was weighted more toward scoping, interpreting results, and judging what a finding actually *means* than toward writing code line-by-line — which feels like a genuine, if unplanned, shift in what "doing" a project like this now involves.